

Project - FileSystem

1. Overview

The goal of this Project is to implement an Ext2 file system on [SUSTechOS](#) and integrate it into the VFS (Virtual File System) and virtio-blk driver, providing a unified interface for user processes above.

Grading breakdown:

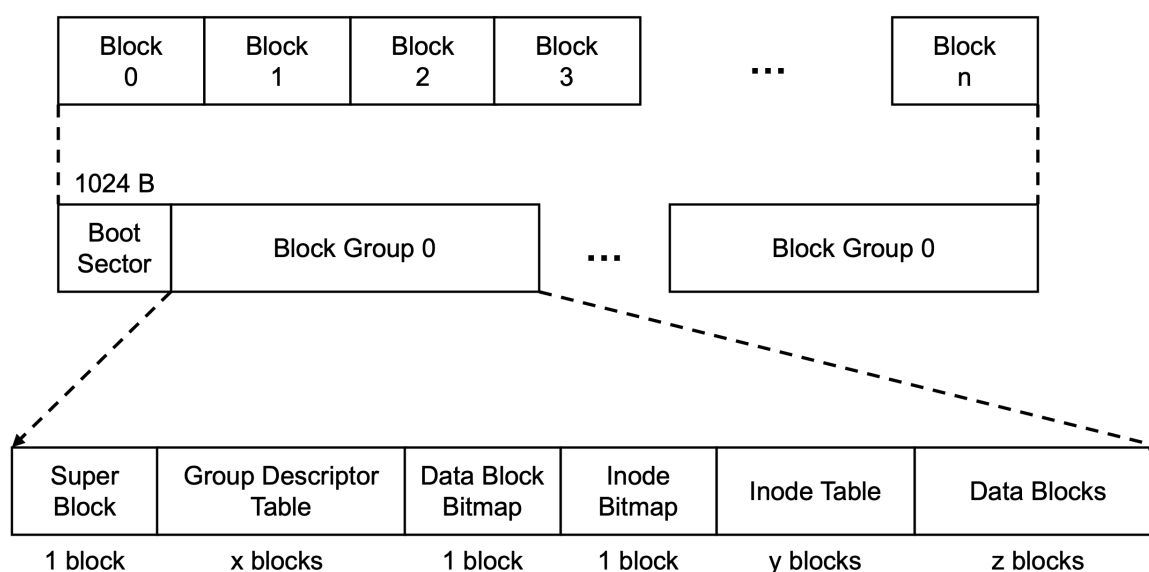
- 15 base points for implementing the Ext2 file system features. The score is determined by checkpoint verification + defense Q&A.
- 5 bonus points, including the journaling system and performance optimization. The score will be determined during the defense Q&A.

2. Introduction to Ext2

2.1 Ext2 Overview and Structure

Ext2 (Second Extended File System) is a classic disk file system introduced by the Linux kernel in 1993. It utilizes a "block group" structure to organize disk space, dividing the entire partition into multiple contiguous block groups. Each block group contains its own independent superblock, Inode table, bitmaps, and data block area. As the cornerstone of the Ext series evolution, Ext2 established the core architecture for its subsequent versions: Ext3 primarily added journaling functionality on top of it to improve crash recovery capabilities; whereas the widely used modern Ext4 further expanded it, supporting larger storage capacities, better performance features (such as extent allocation, multi-block allocation), and richer functionalities.

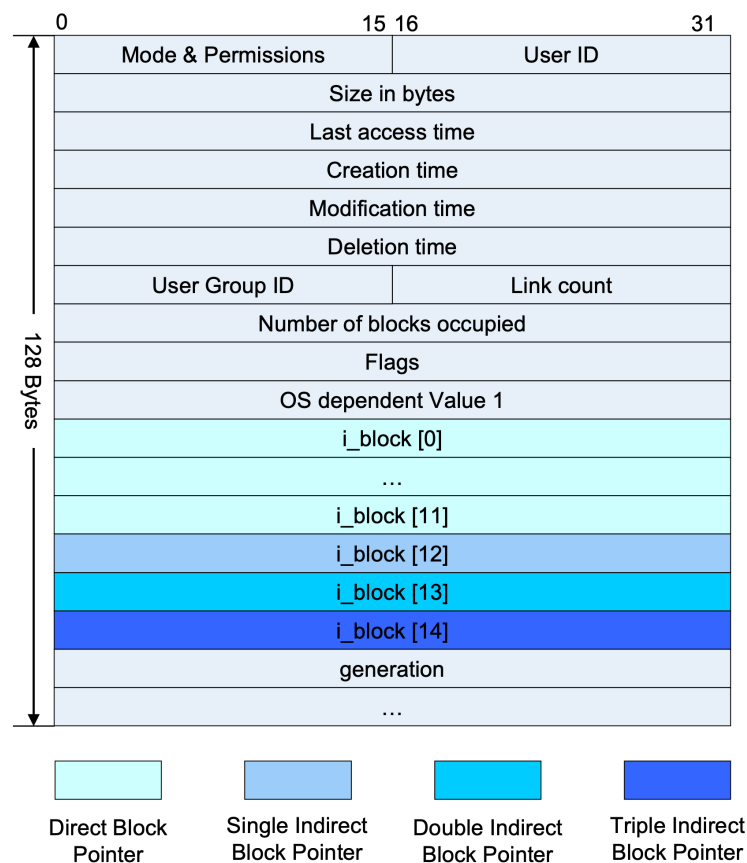
The Ext2 file system uses the "block" as the minimum unit for data storage and allocation. The block size can be configured when the file system is created, typically being 1KB, 2KB, or 4KB. The physical layout of a complete Ext2 file system on disk is shown in the following figure:



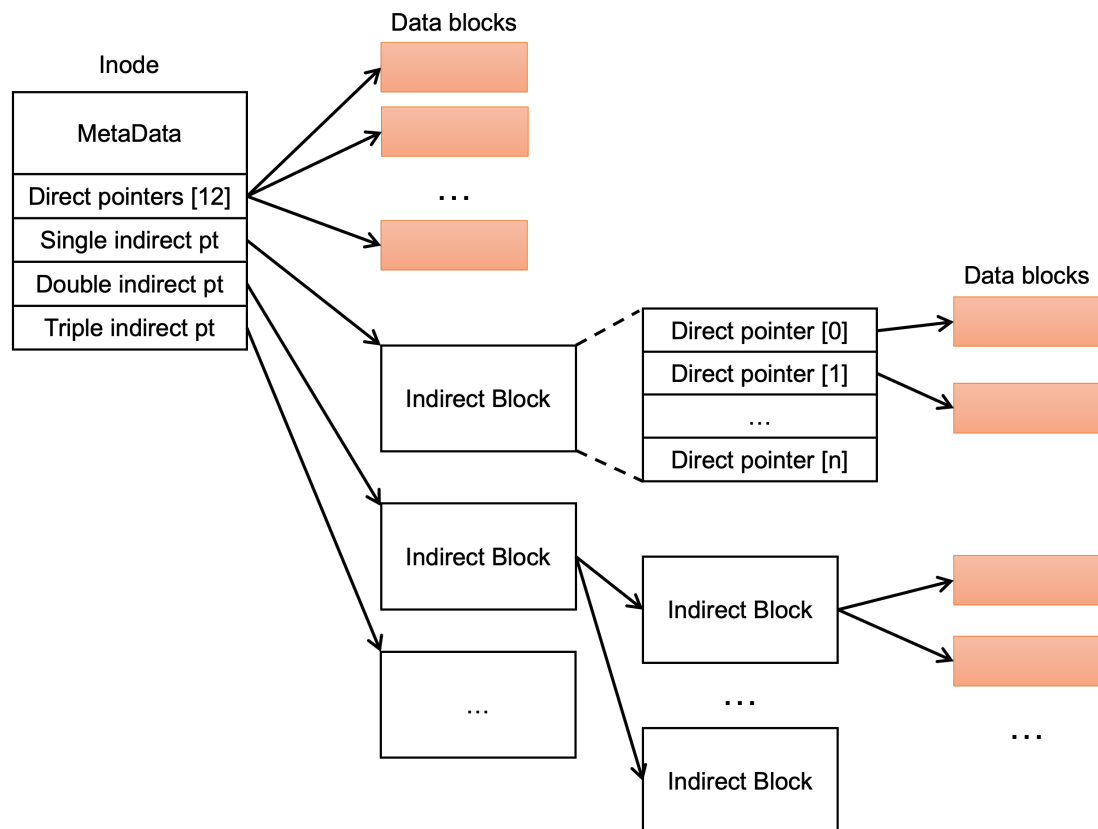
As shown in the figure, the beginning of the Ext2 file system is a boot block, occupying 1024 bytes, followed by multiple continuously arranged block groups. Each block group contains the following parts:

- **Superblock:** Stores the global metadata of the entire file system, such as block size, total number of block groups, free block/Inode counts, etc. The superblock of the first block group (Block Group 0) is located at a fixed offset of 1024 bytes, and the superblocks in the remaining block groups are backups used for disaster recovery.
- **Block Group Descriptor Table:** An array consisting of multiple block group descriptors. Each descriptor records key information about the corresponding block group, such as the starting block number of the block bitmap and Inode bitmap, the starting block number of the Inode table, etc.
- **Block Bitmap:** A bit map used to track the free and used status of all data blocks within this block group (1 bit represents 1 data block).
- **Inode Bitmap:** A bit map used to manage the free status of all Inodes within this block group.
- **Inode Table:** A continuously stored array of Inodes. The size of each Inode is fixed at 128 bytes, used to store the metadata of files or directories.
- **Data Blocks:** The disk space that actually stores file content or directory entry lists.

The core concept among these, the **Inode** (Index Node), is the key data structure for Ext2 to manage file entities. Each Inode is fixed at 128 bytes and contains all the meta-information of a file/directory, such as type, permissions, owner, size, timestamps (creation, modification, access time), and an array of pointers pointing to its data blocks. Its structural diagram is shown below (translated from: [Zhihu](#)):

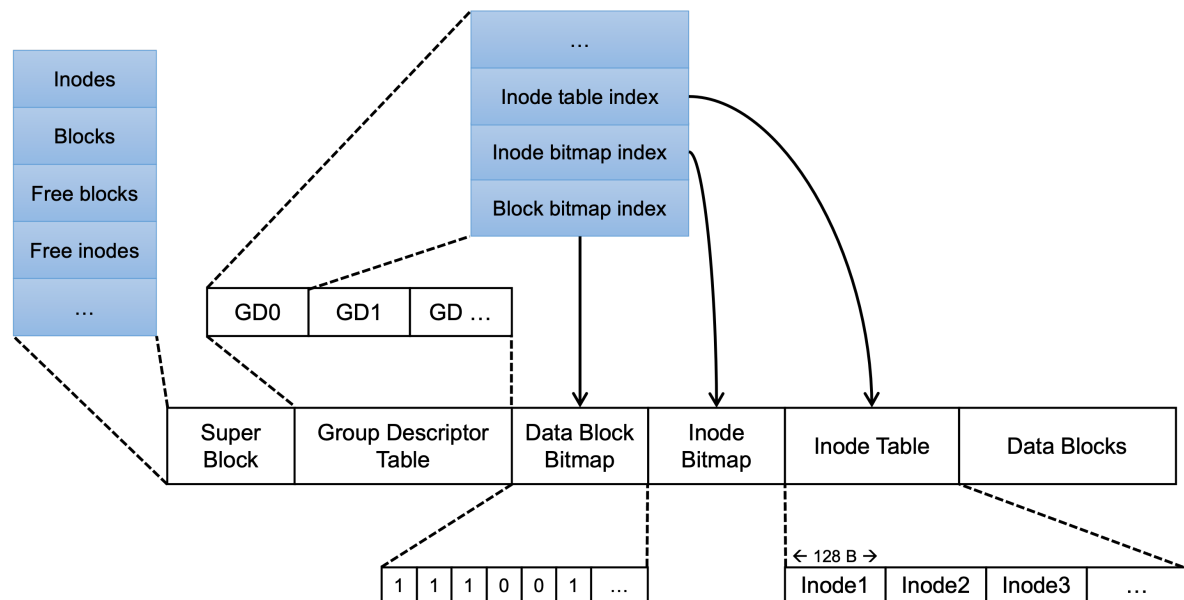


The `i_block[xx]` array at the end of the figure represents the data block pointers. **The first 12 are direct pointers**, pointing directly to the data blocks storing file data; **the 13th is a singly indirect pointer**, pointing to a block containing more direct pointers; **the 14th is a doubly indirect pointer**; **the 15th is a triply indirect pointer**. Its multi-level indexing principle is illustrated in the following figure:



2.2 Ext2 Initialization Process

When the operating system mounts an Ext2 file system, it executes a series of initialization operations to establish its management structures in memory. The structures related to block group initialization in Ext2 are as follows:



The corresponding process is as follows:

- 1. Read the Superblock:** The operating system first reads the **superblock** from a fixed offset on the disk (at the 1024-byte mark of the first block group). It verifies the magic value (0xEF53) within it to confirm that this is a valid Ext2 file system.
- 2. Load the Block Group Descriptor Table:** Based on the information in the superblock, it locates and reads the entire **block group descriptor table** into memory. This table provides key metadata for all block groups, such as the locations of bitmaps and Inode tables.
- 3. Establish Cache:** Caches the superblock and block group descriptors into memory, and indexes to the block bitmap, Inode bitmap, and Inode table based on the block group

descriptors.

2.3 Directory Storage and Indexing

In Ext2, a **Directory is a special type of file**, whose Inode type is marked as `S_IFDIR`. The structure and indexing of a directory are shown in the figure:

When a user wants to check if a file exists (e.g., `/home/user/file.txt`), the operating system indexes to its Inode through the following steps:

1. **Path Resolution:** Starts from the root directory (defaults to Inode 2). Reads the root directory's Inode, and finds the data block storing the root directory entries based on its `i_block[]` pointers.
2. **Directory Entry Traversal:** Linearly scans the directory entries within the root directory's data block, searches for the entry named `"home"`, and obtains its Inode number.
3. **Recursive Search:** Reads the Inode of the `"home"` directory, then reads its data block, searches for the directory entry named `"user"`, and obtains its Inode number. This process is repeated until the directory entry named `"file.txt"` is found in the data block of the `"user"` directory, acquiring the **Inode number** of the target file.

2.4 Ext2 Read and Write Operations

The file read operation in Ext2 is relatively simple: after indexing to the target file through the directory, the file data can be read using `i_block[xx]`.

The file write operation in Ext2 is slightly more complex: if **file size expansion is not needed**, it only requires overwriting the content of the target data block and updating the metadata of the single Inode; if **file size expansion is needed**, it additionally requires finding and allocating free data blocks from the block bitmap, as well as updating global metadata (such as the superblock, etc.).

2.5 Evolution from Ext2 to Ext4

As a widely used format in the early days of Linux, while the Ext2 file system was stable and reliable, it gradually showed limitations in performance, reliability, and functionality. To accommodate growing demands, Ext2 underwent two major upgrades: Ext3 and Ext4.

Ext2 --> Ext3

The most core improvement of Ext3 over Ext2 is the introduction of **Journaling functionality**. In Ext2, if the system suddenly loses power or crashes while writing file metadata (e.g., Inodes, directory entries) or actual data, the file system may be left in an inconsistent state due to incomplete partial updates. At this point, a time-consuming full disk check (`fsck`) must be performed to scan and repair errors.

Ext3's journaling mechanism solves this problem. Its principle is: before actually executing disk write operations, the **"intentions"** of these operations are first recorded into a dedicated journal area. This journal acts like an operation checklist. If all operations are completed successfully, the corresponding journal entries are marked as complete and cleared. If a crash occurs, during the next mount, the system only needs to read and analyze the journal, and then Redo or Undo the relevant operations based on the records to quickly and safely restore the file system to a consistent state prior to the crash, thereby avoiding the massive overhead of a full disk scan.

Ext3 provides three journaling modes to balance security and performance:

- **Journal mode:** Records all metadata and file data. It has the highest security but the largest performance overhead.
- **Ordered mode:** Only records metadata, and ensures data blocks are written before metadata. It balances security and efficiency.
- **Writeback mode:** Only records metadata and does not strictly guarantee the write order. It offers the best performance.

Ext3 --> Ext4

Ext4 is a comprehensive upgrade to Ext3, aiming to break through its performance bottlenecks in ultra-large capacity and high-concurrency scenarios.

A core improvement of Ext4 is the introduction of **Extents** to replace the traditional block mapping method in Ext2/3. In Ext2/3, a large file might need to map thousands of data blocks through a massive number of indirect pointer blocks, resulting in bloated metadata and susceptibility to fragmentation. Extents, on the other hand, describe continuous data blocks in a file as a range (e.g., "starting block number 1000, 200 continuous blocks", a concept similar to "base address + length" in memory management), requiring only one extent structure to represent them, thus significantly reducing the amount of metadata.

In addition, Ext4 implements multiple allocation strategies:

- **Multi-block allocation:** Allows multiple continuous data blocks to be allocated at once, reducing the overhead and fragmentation caused by frequent allocation operations.
- **Delayed allocation:** Postpones the actual allocation decision of data blocks until the data is about to be written back to the disk, thereby accumulating more write requests to achieve better continuous allocation.

3. Grading Requirements

Among the grading points this time, passing the checkpoint accounts for only 40%, and the Q&A accounts for 60%.

3.1 Basics

[4 pts] Mounting and Superblock Parsing: The system needs to correctly read the Superblock from a fixed offset on the disk (at the 1024-byte mark of the first block group). Based on the superblock information, correctly load the Block Group Descriptor table into memory and establish caches for bitmaps and Inode tables.

[5 pts] File Creation, Read, and Write: Support basic file operation commands: `read`, `write`, `touch`, and for small file writes, be able to correctly overwrite the target data block content.

3.2 Directories and Indexing

[2 pts] Directory Creation, Lookup, and Deletion:

- Support directory operations such as `mkdir`, `rmdir`, `get_dents`.
- Correctly identify directory files marked with the `S_IFDIR` type, and support path resolution starting from the root directory (Inode 2).
- Be able to read directory data blocks to locate the target file's Inode number.

[2 pts] Direct and Indirect Indexing:

- Implement parsing of the `i_block[xx]` pointer array; must support data read and write for the first 12 direct pointers.

- Support reading and writing of large files, correctly handling the block mapping principles of the 13th (singly), 14th (doubly), and 15th (triply) indirect pointers.

3.3 Metadata and System Specifications

[1 pts] Correct Updates of Metadata:

- When writing to a file or extending file size, key metadata in the Inode (such as file size, access time, modification time, etc.) must be updated correctly and promptly.
- When file extension is involved, operations to allocate free data blocks from the block bitmap must be executed correctly, and global metadata such as the superblock must be updated synchronously.

[1 pts] Interface Specifications:

- Needs to integrate into the system's VFS. The implemented Ext2 must not only be able to read and write regular files but also support special files of the FIFO (Named PIPE) type within the system.

4. Bonus (Total 5 Bonus Points)

This section aims to encourage students to explore advanced features of modern file systems and optimize file system performance.

4.1 Journaling System to Ext4 (1 pts)

Evolve the basic Ext2 file system towards Ext3 or Ext4 by implementing a basic journaling system: introduce a Journaling area to record the intentions of disk operations.

Note: Developing towards Ext4 based on the journaling system can also provide certain optimizations for subsequent performance testing, such as introducing Extents structures to replace massive indirect pointer block mappings, reducing metadata fragmentation; or implementing multi-block continuous allocation and delayed allocation strategies to lower the overhead caused by frequent allocations.

4.2 Performance Testing (4 pts)

This Bonus item will undergo performance testing during the final defense, and specific scores will be awarded based on ranking (maximum 4 points), including the following test points:

- **Large file throughput:** Test the sequential read and sequential write performance of data.
- **Random I/O performance:** Test the random read and random write performance of data.
- **Metadata and small file operations:** Test the latency performance of small file creation, writing, reading, and deletion.

5. Defense and Submission Requirements

The final defense requires demonstrating the implementation ideas and details in the Project. We will ask questions about certain parts of your code. It is expected to be scheduled in Week 15 (June 1st ~ June 5th). **The defense will not be conducted in the form of PPTs, but through on-site Q&A. Passing the checkpoint only accounts for 40% of the score, and the defense Q&A accounts for 60%. Please make sure you fully understand the code logic.**

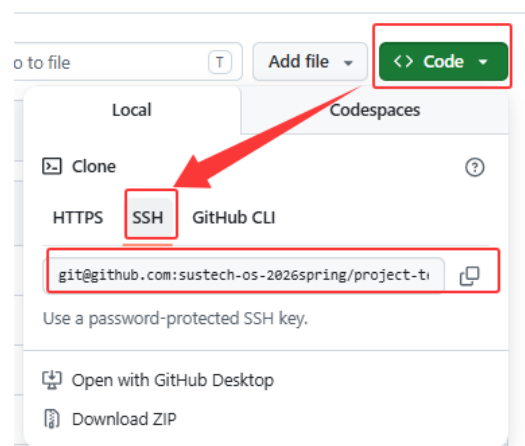
This semester, GitHub Classroom will be used as the code repository (invitation link: <https://classroom.github.com/a/JGNFCPZR>). Students who haven't registered on GitHub, please create an account, enter the classroom, and bind your student ID. If your student ID is already bound, please contact the TA. In addition, please make sure to bind your student ID email to GitHub and set it as a public email. We will check the binding status of student IDs and GitHub accounts during the defense week Q&A:



Apart from the code repository, please upload a report in PDF format on Blackboard as a group. The report needs to include the list of group members, project design architecture, implementation details, and screenshots of passed tests.

We have provided the basic code in the repository. All students are required to complete the project based on this template code. The operation procedures are as follows:

1. Join the classroom via the invitation link and select your corresponding student ID.
Invitation link: <https://classroom.github.com/a/JGNFCPZR>
2. One member of each team shall create the team, and the remaining members shall join it. Each team must consist of exactly three members. In principle, all three members should be from the same lab class. If any member is from a different lab class, they must provide a class schedule certificate from the tis system to prove that they have no conflicting courses during the class hours of the team's lab class.
3. A dedicated team repository will be automatically generated after team members join the group.
4. Configure the SSH key in the Ubuntu experimental environment by following the instructions in the **GitHub_SSH_Key.pdf** document.
5. Click **Code > SSH** in the upper-right corner of the repository page to copy the Git clone link. Run the `git clone` command in the experimental environment to download the repository code to your local device.



6. Read the README file in the repository and finish the project tasks.
7. Commit and push your final code to the team repository, and submit your report in PDF format via the BB site. The BB submission portal will be opened after team formation is completed.

AI Usage Guidelines

This course **encourages** the use of various generative AI tools (such as ChatGPT, Gemini, DeepSeek, etc.) to assist learning and development while completing the Project, but please comply with the following guidelines:

- **Code Generation Declaration:** If AI tools were used to generate code within your project, a declaration must be made upon submission.
- **Submission Accompanying Document:** Please create a document named `AI_USAGE.md` (or `.pdf`) in the project root directory and include the corresponding contextual chat records (including Prompts and the AI's Responses) when generating that piece of code.
- **Code Comprehension:** Large language models are highly prone to "hallucinations" or logical loopholes when generating operating system code. **Please thoroughly review the AI-generated code to ensure you understand how it works. Furthermore, if you cannot explain the code you submitted during the code review at the final defense, you will face the risk of point deduction. The final score will be determined jointly by the Q&A and testing, with the Q&A being a significant consideration.**

Note:

1. Declaring the use of AI tools will not affect the final grade. There will be no grading difference between handwritten code and AI-generated code, but pay attention to code comprehension.
2. The AI requirements for the Project are different from the assignment requirements.